

DistilBERT frente a BERT en clasificación de texto

Desempeño, eficiencia y un estudio de ablación sobre tres datasets

Proyecto 1 — Curso de Procesamiento de Lenguaje Natural

Resumen

Se hizo *fine-tuning* de BERT-base y DistilBERT-base sobre tres tareas de clasificación de texto (SST-2, AG News y Yelp Polarity) bajo un protocolo idéntico, y se comparó a los dos modelos en desempeño y en eficiencia. Sobre esa base se ejecutó un estudio de ablación de seis configuraciones que varía la cabeza de clasificación y qué parte del *transformer* se entrena.

Los resultados sostienen tres afirmaciones. Primera: con el 61 % de los parámetros, DistilBERT conserva entre el 98,9 % y el 99,5 % de la *accuracy* de BERT, y la brecha depende de la tarea — se abre en análisis de sentimiento sobre frases cortas y se cierra hasta desaparecer en clasificación de temas. Segunda: medida en condiciones comparables, la inferencia de DistilBERT es exactamente $2\times$ más rápida en los nueve puntos de una rejilla de *batch* y longitud, que es lo que predice pasar de 12 capas a 6. Tercera: la arquitectura de la cabeza de clasificación es irrelevante — cinco configuraciones distintas caben en 0,0014 de F1 — mientras que congelar el *encoder* cuesta entre 3,4 y 8,4 puntos de F1 según la tarea.

En total son 24 corridas de entrenamiento, unas 2,5 horas de GPU en una NVIDIA RTX A6000. Todas las métricas de este informe se generan automáticamente desde los archivos JSON versionados en el repositorio.

Índice

Resumen	1
1. Introducción y objetivos	2
2. Datos	2
2.1. Normalización	3
2.2. Dos decisiones sobre los splits	3
3. Metodología	3
3.1. Los modelos y qué se hizo con los pesos	3
3.2. La cabeza de clasificación del estudio de ablación	4
3.3. Protocolo de entrenamiento	4
3.4. Métricas	5
4. Resultados de desempeño	5
4.1. Comparación global	5
4.2. Curvas de entrenamiento	6
4.3. Análisis por clase	6
5. Estudio de ablación	7
5.1. Diseño	7
5.2. Resultados	7
5.3. Lectura	7
6. Eficiencia	9
6.1. Tamaño y memoria	9
6.2. Por qué la latencia hubo que medirla aparte	9
6.3. Latencia en condiciones controladas	9
6.4. Las tres dimensiones a la vez	10

7. La configuración ganadora en los dos modelos	10
8. Limitaciones	11
9. Conclusiones	12
A. Reproducibilidad	13
A.1. Estructura del repositorio	13
A.2. Cómo se reproduce	13
A.3. Trazabilidad de las cifras	14
A.4. Regenerar las figuras sin GPU	14
A.5. Compilar este informe	14

1. Introducción y objetivos

BERT [1] estableció el procedimiento estándar para tareas de clasificación de texto: pre-entrenar un *transformer* bidireccional sobre texto no etiquetado y después ajustarlo a la tarea concreta. El costo de ese procedimiento, sin embargo, es alto en inferencia, que es donde un modelo pasa la mayor parte de su vida útil. DistilBERT [2] responde a ese costo con destilación de conocimiento: un modelo de 6 capas entrenado para imitar a uno de 12, con la promesa de conservar la mayor parte del desempeño a la mitad del tamaño.

Este proyecto pone esa promesa a prueba con una condición que se respeta en todo el trabajo: **la única diferencia entre las dos columnas de cualquier tabla debe ser el modelo.** Mismo código, mismos datos, mismo optimizador, misma semilla, misma GPU. Cuando esa condición no se puede cumplir — y hay un caso en el que no se cumple, la latencia medida al final de cada entrenamiento — se dice explícitamente y se mide de nuevo en condiciones controladas (Sección 6).

Los objetivos concretos son cuatro:

1. Ajustar ambos modelos a tres tareas de clasificación de distinta naturaleza y tamaño, bajo un protocolo idéntico.
2. Reportar el desempeño con *accuracy*, *precision*, *recall* y F1, y la eficiencia con número de parámetros, latencia de inferencia y memoria de GPU.
3. Determinar, mediante un estudio de ablación, qué partes del modelo contribuyen realmente al desempeño: la arquitectura de la cabeza de clasificación o la adaptación del *encoder*.
4. Aplicar la configuración ganadora a ambos modelos y comparar.

2. Datos

Se eligieron tres tareas que varían en número de clases, longitud de texto y tamaño, para que las conclusiones no dependan de un único régimen: SST-2 [3], distribuida dentro del *benchmark* GLUE [4], y AG News y Yelp Polarity, ambas en la recopilación de Zhang et al. [5].

Dataset	Fuente (HuggingFace)	Clases	Train	Val.	Test	max_length
SST-2	nyu-mll/glue, conf. sst2	2	60 614	6 735	872	64
AG News	fancyzhx/ag_news	4	108 000	12 000	7 600	128
Yelp Polarity	fancyzhx/yelp_polarity	2	100 000*	20 000*	38 000	256

Tabla 1: Los tres datasets, ya normalizados por `src/data_adapter.py`. (*) Yelp va submuestreado; véase más abajo.

2.1. Normalización

Los tres datasets llegan del Hub con esquemas distintos: SST-2 llama `sentence` a su columna de texto y arrastra una columna `idx`; AG News y Yelp usan `text`; ninguno de los tres trae los mismos splits. El módulo `src/data_adapter.py` los reduce a una forma única — un `DatasetDict` con splits `train/validation/test` y solo las columnas `text` (str) y `label` (int) — de modo que el resto del *pipeline* no sabe con qué dataset trabaja. Añadir una tarea nueva es añadir una entrada a un registro, no tocar el entrenamiento.

2.2. Dos decisiones sobre los splits

Ambas afectan a cómo deben leerse los números, así que conviene dejarlas explícitas.

El test de SST-2 no se usa. En GLUE ese split viene sin etiquetas (`label = -1`), porque la evaluación oficial es en servidor. Se usa entonces el `validation` oficial (872 frases) como conjunto de test, y se aparta un 10 % del train como validación propia. Es el protocolo habitual en la literatura, y es lo que hace que nuestros números sean comparables con los publicados. El adaptador verifica explícitamente que ningún split contenga la etiqueta `-1` y aborta si la encuentra: es la clase de error que, si pasa inadvertido, produce métricas sin sentido sin lanzar ninguna excepción.

Yelp va submuestreado. Se usan 100 000 de las 560 000 reseñas de train y 20 000 de validación. Entrenar con el conjunto completo a 256 *tokens* multiplicaría por cinco el costo para mover la *accuracy* algunas décimas. Lo importante para la validez de la comparación es que **el mismo presupuesto se aplica a los dos modelos**: el submuestreo usa la misma semilla y produce exactamente el mismo subconjunto para BERT y para DistilBERT.

En los dos datasets que no traen un split de validación etiquetado (AG News y Yelp), la validación es un 10 % del train separado con semilla fija. El conjunto de test nunca participa en ninguna decisión.

3. Metodología

3.1. Los modelos y qué se hizo con los pesos

Esta subsección es deliberadamente explícita, porque el tratamiento de los pesos pre-entrenados es donde un experimento de *fine-tuning* se invalida con más facilidad y sin dar ningún error.

Origen. Los dos modelos se construyen a partir de los *checkpoints* públicos de HuggingFace, que se enumeran en la Tabla 2. DistilBERT es una destilación del primero de ellos: conserva una de cada dos capas, elimina el *pooler* y los *embeddings* de segmento, y mantiene la dimensión oculta (768) y el vocabulario. Los pesos se descargan del Hub en el momento de construir el modelo; **no hay ningún peso almacenado en el repositorio**, ni versionado, ni modificado, ni redistribuido. El repositorio contiene únicamente código, métricas en JSON y figuras.

Nombre corto	Checkpoint en el Hub	Capas	Parámetros
BERT-base	<code>bert-base-uncased</code>	12	109 483 778
DistilBERT-base	<code>distilbert-base-uncased</code>	6	66 955 010

Tabla 2: Los dos *checkpoints* de partida. El número de parámetros corresponde a una cabeza de dos clases; con las cuatro clases de AG News sube en 1 538 en ambos modelos ($768 \times 2 + 2$ pesos de salida adicionales).

Qué se inicializa al azar. Al construir un clasificador, el *encoder* llega pre-entrenado pero la cabeza de clasificación se inicializa aleatoriamente: es la única parte que no ha visto nunca datos. El aviso “*newly initialized*” que imprime `transformers` es esperado y correcto; su ausencia sería la señal de alarma. La semilla 42 fija esa inicialización, de modo que la cabeza de BERT y la de DistilBERT parten del mismo estado aleatorio.

Qué se actualiza durante el entrenamiento. En las corridas base se actualizan todos los parámetros: *encoder* y cabeza. En las configuraciones congeladas del estudio de ablación se pone `requires_grad = False` sobre el *encoder* y, además, esos parámetros **no se pasan al optimizador**. Los dos pasos son necesarios: con solo el primero, AdamW seguiría aplicando *weight decay* a pesos que no reciben gradiente, y el modelo se degradaría de forma silenciosa.

El tokenizador se deriva siempre del modelo. Cada *checkpoint* trae su propio vocabulario. Usar el tokenizador de otro modelo rompe la correspondencia entre *token* e *embedding* y el modelo predice ruido sin lanzar ninguna excepción. Por eso `build_tokenizer()` nunca acepta un vocabulario suelto: lo deriva del identificador del modelo.

Qué se conserva y qué no. Durante el entrenamiento se mantiene en memoria una copia del *state dict* de la época con mejor F1 de validación, y es esa copia la que se carga antes de evaluar en test. Al terminar, el modelo ajustado **se descarta**: no se escribe en disco ni se versiona. Todo lo que sobrevive a una corrida son sus métricas. Es una decisión consciente — los *checkpoints* ajustados pesan cientos de megabytes cada uno y son reproducibles desde el código y la semilla — y tiene la consecuencia práctica de que cargar de nuevo cualquier modelo del proyecto parte siempre de los pesos originales del Hub, sin estados intermedios que puedan haberse corrompido.

3.2. La cabeza de clasificación del estudio de ablación

Las clases `AutoModelForSequenceClassification` de HuggingFace traen una cabeza fija, y — esto es lo que obliga a no usarlas aquí — una cabeza **distinta en cada arquitectura**: BERT pasa el *token* [CLS] por un *pooler* con `tanh`; DistilBERT usa un `pre_classifier` con `ReLU`. Comparar cabezas montadas sobre dos preprocesos distintos mezclaría dos variables en el mismo experimento.

Por eso el estudio de ablación usa una envoltura propia (`ClasificadorConCabeza` en `src/models.py`): *encoder* desnudo (`AutoModel`) más un MLP construido por nosotros sobre el estado oculto del *token* [CLS], idéntico para las dos arquitecturas. Así la única diferencia entre dos corridas es lo que el experimento cambia a propósito.

3.3. Protocolo de entrenamiento

El bucle de entrenamiento está escrito a mano, sin el `Trainer` de HuggingFace, para tener control explícito sobre el *scheduler*, el recorte de gradientes y el registro por iteración. Los hiperparámetros son idénticos para los dos modelos y las tres tareas.

Dos detalles del bucle que cambian los resultados si se omiten. El primero: el *weight decay* no se aplica a los términos de *bias* ni a los de `LayerNorm`, porque son parámetros de escala y desplazamiento y penalizarlos degrada el modelo. El segundo: con precisión mixta hay que des-escalar los gradientes *antes* de recortarlos (`scaler.unscale_()` antes de `clip_grad_norm_()`), o el umbral se aplicaría sobre gradientes escalados y no significaría lo que se cree.

El test se mira una sola vez. La configuración ganadora del estudio de ablación se elige por F1 de validación. El conjunto de test se evalúa al final, con el *checkpoint* ya seleccionado, y no participa en ninguna decisión. Elegir sobre el conjunto que después se reporta es la forma más común de producir números optimistas.

Optimizador	AdamW, $1r = 2 \times 10^{-5}$, <code>weight_decay</code> = 0,01
<i>Weight decay</i>	No se aplica a <i>bias</i> ni a <code>LayerNorm</code>
<i>Scheduler</i>	<i>Warmup</i> lineal 10% + decaimiento lineal
Épocas / <i>batch</i>	2 / 32
Precisión	fp16 (<i>mixed precision</i>)
Recorte	Norma máxima de gradiente = 1,0
Selección	<i>Checkpoint</i> con mejor F1 de validación
Semilla	42 en Python, NumPy y PyTorch

Tabla 3: Protocolo común. El *learning rate* está dentro del rango recomendado en el artículo original de BERT (5×10^{-5} , 3×10^{-5} o 2×10^{-5}).

3.4. Métricas

Precision, *recall* y F1 se reportan como promedio macro: se promedia la métrica de cada clase dándoles el mismo peso. Es lo adecuado con clases desbalanceadas (SST-2 está en 56/44) y, además, el promedio micro en clasificación multiclase de etiqueta única colapsa a la *accuracy* y no aportaría información.

Para la eficiencia se miden parámetros, tamaño en fp32, latencia de inferencia y pico de memoria de GPU. La medición de latencia toma dos precauciones que cambian el resultado por completo: un *warm-up* de varias pasadas — las primeras incluyen la carga de *kernels* CUDA y la reserva de memoria, y medirlas infla la cifra varias veces — y una llamada a `torch.cuda.synchronize()` en cada iteración, porque las llamadas a CUDA son asíncronas y sin sincronizar se mediría el tiempo de *encolar* la operación, no el de ejecutarla. Para la memoria se resetea el contador de pico antes de medir, o se arrastraría el pico del entrenamiento previo.

4. Resultados de desempeño

4.1. Comparación global

La Tabla 4 recoge las seis corridas base. Todas las cifras salen de los JSON versionados en `results/metrics/`.

Dataset	Modelo	Accuracy	Precision	Recall	F1	Train (min)
SST-2	BERT-base	0,9232	0,9232	0,9231	0,9231	4,2
SST-2	DistilBERT-base	0,9128	0,9134	0,9125	0,9127	2,7
AG News	BERT-base	0,9441	0,9443	0,9441	0,9441	9,1
AG News	DistilBERT-base	0,9450	0,9452	0,9450	0,9450	5,5
Yelp	BERT-base	0,9652	0,9652	0,9652	0,9652	16,0
Yelp	DistilBERT-base	0,9606	0,9606	0,9606	0,9606	8,2

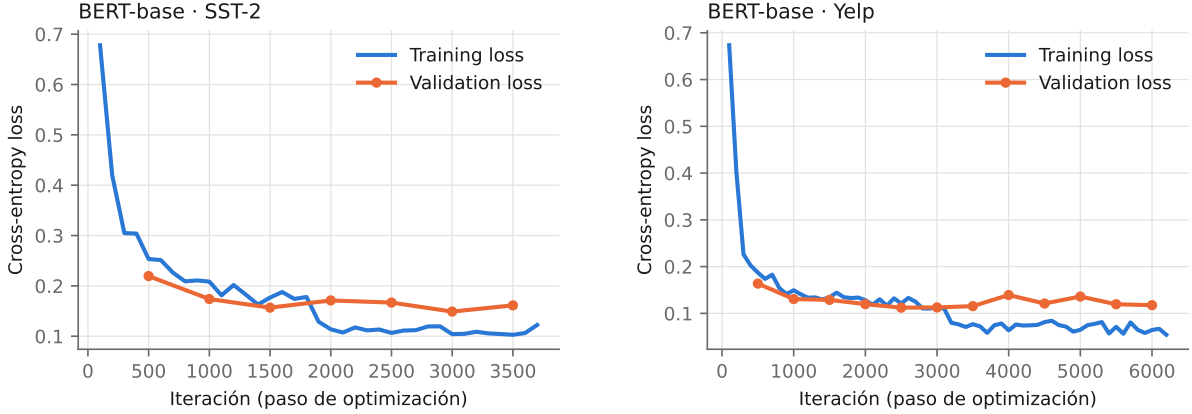
Tabla 4: Desempeño en test. *Precision*, *recall* y F1 son promedio macro. Mismo protocolo en las seis corridas: 2 épocas, *batch* 32, $1r$ 2×10^{-5} , fp16, RTX A6000.

DistilBERT conserva el 98,9% de la *accuracy* de BERT en SST-2 y el 99,5% en Yelp, con el 61% de los parámetros. En AG News la diferencia cambia de signo (0,09 puntos para DistilBERT), pero conviene no sobreinterpretarla: sobre 7 600 ejemplos de test, 0,09 puntos son **siete ejemplos**. La lectura honesta es un empate, no una victoria.

Lo que sí es un patrón es la *dependencia de la tarea*: la brecha entre los dos modelos es de 1,04 puntos en SST-2, 0,46 en Yelp y esencialmente nula en AG News. Clasificar temas de noticias no necesita las seis capas adicionales; el análisis de sentimiento sobre frases cortas — con negaciones, ironía y contraste — sí las aprovecha.

Como referencia externa, el valor de DistilBERT en SST-2 (91,3) cae donde caen los publicados, lo que sugiere que el protocolo está bien calibrado y que no hay fugas entre splits.

4.2. Curvas de entrenamiento



(a) BERT en SST-2: 37 registros, 2 épocas.

(b) BERT en Yelp: 62 registros sobre 100 000 ejemplos.

Figura 1: *Cross-entropy loss* de entrenamiento y de validación frente al número de iteraciones. La validación se evalúa cada 500 pasos, de ahí que tenga menos puntos. Las curvas de las seis corridas están en `results/figures/`.

Las curvas (Figura 1) muestran el comportamiento esperado de un *fine-tuning* corto: la *loss* de entrenamiento cae con rapidez en los primeros cientos de pasos y después se aplana, mientras la de validación se estabiliza. En SST-2, el dataset más pequeño, la validación empieza a separarse del entrenamiento hacia el final de la segunda época — el inicio del sobreajuste —, que es exactamente la razón por la que el protocolo conserva el *checkpoint* de la mejor época de validación en lugar del último.

4.3. Análisis por clase

El promedio macro oculta dónde se equivoca cada modelo. En AG News, la única tarea con más de dos clases, los errores no se reparten de forma uniforme (Tabla 5).

Clase	BERT-base			DistilBERT-base		
	P	R	F1	P	R	F1
World	0,958	0,955	0,956	0,963	0,952	0,957
Sports	0,987	0,985	0,986	0,987	0,990	0,988
Business	0,927	0,903	0,915	0,924	0,905	0,915
Sci/Tech	0,905	0,934	0,919	0,906	0,933	0,920

Tabla 5: Desempeño por clase en AG News (1900 ejemplos de test por clase).

El patrón es idéntico en los dos modelos. *Sports* es casi perfecta ($F1 \approx 0,99$): su vocabulario es distintivo y apenas se solapa con el resto. *Business* y *Sci/Tech* son las dos peores, y la matriz de confusión explica por qué: BERT clasifica 140 noticias de *Business* como *Sci/Tech* y 90 en sentido contrario; DistilBERT comete 140 y 92. Esos dos flujos concentran la mayor parte del error de ambos modelos.

La interpretación es que el límite no está en la capacidad del modelo sino en la tarea: una noticia sobre los resultados trimestrales de una empresa de semiconductores pertenece legítimamente a las dos categorías. Que BERT, con el doble de capas, cometa *el mismo* número de

confusiones *Business* $\rightarrow$ *Sci/Tech* que DistilBERT es la evidencia más clara de este informe de que la capacidad adicional no se está desaprovechando: sencillamente no hay nada más que extraer de esos ejemplos.

5. Estudio de ablación

5.1. Diseño

El estudio se hace sobre DistilBERT y varía dos cosas dejando todo lo demás fijo: la arquitectura de la cabeza de clasificación y qué parte del *transformer* se entrena. Son seis configuraciones sobre los tres datasets, es decir 18 corridas.

Config	Cabeza	Encoder	Eje que prueba
lineal	$768 \rightarrow C$	entrenable	menos capas (ninguna oculta)
h128	$768 \rightarrow 128 \rightarrow C$	entrenable	menos neuronas
h768	$768 \rightarrow 768 \rightarrow C$	entrenable	referencia
h2048	$768 \rightarrow 2048 \rightarrow C$	entrenable	más neuronas
c2	$768 \rightarrow 512 \rightarrow 256 \rightarrow C$	entrenable	más capas
frz	$768 \rightarrow 768 \rightarrow C$	congelado	congelar el <i>transformer</i>

Tabla 6: Las seis configuraciones. C es el número de clases. El diseño cubre los *dos* sentidos de cada eje — más y menos capas, más y menos neuronas — porque con una sola dirección no se puede afirmar si la métrica sube o baja de forma monótona.

Una corrección necesaria en la configuración congelada. La configuración **frz** se entrena con $\text{lr} = 10^{-3}$ en vez de 2×10^{-5} . El *learning rate* pequeño está pensado para ajustar pesos ya pre-entrenados; la cabeza sobre un *encoder* congelado se aprende desde cero y con 2×10^{-5} apenas avanzaría. Sin este ajuste, **frz** perdería por el *learning rate* y no por estar congelada, que es lo que se quiere medir.

5.2. Resultados

Config	SST-2	AG News	Yelp	Media	Params entren.	Tiempo (min)
lineal	0,9483	0,9462	0,9603	0,9516	66,4 M	15,3
h2048	0,9470	0,9459	0,9601	0,9510	67,9 M	16,3
h128	0,9480	0,9454	0,9593	0,9509	66,5 M	15,9
c2	0,9476	0,9434	0,9604	0,9505	66,9 M	15,9
h768	0,9485	0,9427	0,9596	0,9503	67,0 M	16,3
frz	0,8574	0,9084	0,9005	0,8887	0,6 M	6,0

Tabla 7: F1 de **validación** por configuración, ordenado por la media de los tres datasets. El tiempo es el total de las tres corridas. La elección se hace con esta columna; el test no participa.

5.3. Lectura

La cabeza de clasificación casi no importa. Las cinco configuraciones con el *encoder* entrenable caben en 0,0014 de F1. Un clasificador lineal iguala a un MLP de 2048 neuronas y a uno de dos capas ocultas. Con conjuntos de validación de entre 6 735 y 20 000 ejemplos, esas diferencias están dentro del ruido: ni siquiera el orden entre ellas es estable — **h768** es la mejor en SST-2 y la peor en media.

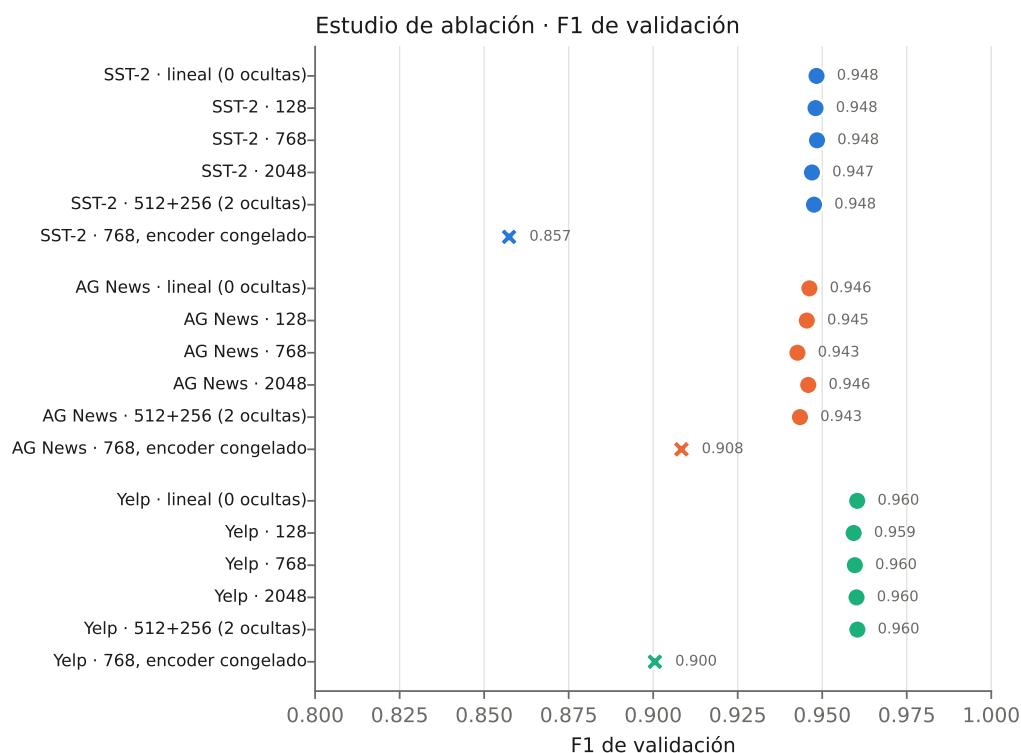


Figura 2: F1 de validación de cada configuración, por dataset. El eje llega desde 0,80 a propósito: recortarlo alrededor de las diferencias haría parecer enormes unas distancias de milésimas. Lo que se ve así es lo que realmente ocurre — las cinco cabezas apiladas en el mismo punto y el *encoder* congelado descolgado —, y esa es la conclusión del estudio.

Esto tiene una explicación razonable. El *encoder* ya produce, en el *token* [CLS], una representación de 768 dimensiones ajustada a la tarea durante el propio *fine-tuning*. Si esa representación es linealmente separable, añadir capas por encima no aporta nada: la capacidad que importa está en el *transformer*, no en el clasificador.

Congelar el *transformer* sí importa, y cuánto depende de la tarea. El *encoder* congelado pierde 8,4 puntos de F1 en SST-2, 5,8 en Yelp y solo 3,4 en AG News. La asimetría es informativa: clasificar temas de noticias se resuelve casi con las características que DistilBERT ya trae de fábrica — es, en buena medida, detección de vocabulario —, mientras que detectar sentimiento en frases cortas con negación y sarcasmo exige que el *encoder* se adapte.

Vale la pena señalar el otro lado del intercambio: **frz** entrena 0,6 M de parámetros en lugar de 66,4 M y tarda 6,0 minutos en lugar de 15,3. Para AG News, donde el costo son 3,4 puntos de F1, es un intercambio que podría ser razonable si el presupuesto de entrenamiento fuera el factor limitante.

Gana lineal, pero no por ser mejor. Queda empatada con las demás dentro del ruido, y es la más pequeña y la más rápida de entrenar. Ante un empate estadístico, la configuración más simple es la elección defendible.

6. Eficiencia

6.1. Tamaño y memoria

Las cifras que no dependen de cómo se mida — número de parámetros, tamaño en disco, número de capas — se resumen en la Tabla 8. Son las que justifican por qué se esperaría que DistilBERT fuese más barato; las dos subsecciones siguientes comprueban si esa expectativa se cumple en tiempo de ejecución.

	BERT-base	DistilBERT-base	Diferencia
Parámetros	109 483 778	66 955 010	−38,8 %
Tamaño en fp32	417,7 MB	255,4 MB	−38,8 %
Capas <i>transformer</i>	12	6	−50 %
Memoria GPU en inferencia	≈1 720 MB	≈1 060 MB	−38 %
Tiempo de entrenamiento (3 datasets)	29,3 min	16,4 min	−44 %

Tabla 8: Métricas estáticas de eficiencia. La reducción de parámetros (38,8 %) es menor que la de capas (50 %) porque la matriz de *embeddings*, que no se reduce, representa una fracción importante del total.

6.2. Por qué la latencia hubo que medirla aparte

Cada corrida de *fine-tuning* guarda un campo `latencia_ms_media`, medido al terminar ese entrenamiento y con la longitud de secuencia de *su* dataset. Esos números **no son comparables entre sí**, y al ponerlos juntos producen un resultado absurdo: BERT en SST-2 marca 3,16 ms y DistilBERT en SST-2 marca 3,50 ms. El modelo con el doble de capas saldría “más rápido”.

Detectar esto y no reportarlo sin más es parte del resultado. La causa, como se ve enseguida, es que con *batch* = 1 la GPU no está calculando sino esperando a que se lancen los *kernels*, de modo que la medida no refleja la arquitectura sino el ruido del lanzamiento. La solución fue un módulo aparte (`src/benchmark.py`) que mide a los dos modelos en la **misma rejilla** de (*batch*, longitud), en la misma GPU y en el mismo proceso. La latencia no depende de los pesos ajustados sino de la arquitectura, así que se mide sobre los *checkpoints* pre-entrenados con entradas sintéticas.

6.3. Latencia en condiciones controladas

Batch	Long.	BERT (ms)	DistilBERT (ms)	Acel.	BERT (MB)	DistilBERT (MB)	Ratio mem.
1	64	5,56	2,60	2,14×	429,9	268,6	0,62×
1	128	5,61	2,61	2,15×	433,0	271,6	0,63×
1	256	5,11	2,74	1,86×	436,1	274,8	0,63×
8	64	7,72	4,01	1,93×	445,9	284,5	0,64×
8	128	13,62	6,95	1,96×	461,5	299,5	0,65×
8	256	26,59	13,39	1,99×	493,9	332,5	0,67×
32	64	24,97	12,62	1,98×	493,9	332,5	0,67×
32	128	49,85	25,10	1,99×	559,9	398,6	0,71×
32	256	98,10	49,25	1,99×	692,0	530,6	0,77×

Tabla 9: Latencia y pico de memoria en la misma rejilla (RTX A6000, fp32, mismo proceso). DistilBERT es 2× más rápido en los nueve puntos.

Medido así, DistilBERT es **2×** más rápido en los nueve puntos de la rejilla, que es exactamente lo que predice pasar de 12 capas a 6. En rendimiento, con *batch* 32 y 64 *tokens*: 2 535 frente a 1 282 muestras por segundo.

Latencia de inferencia en la misma GPU

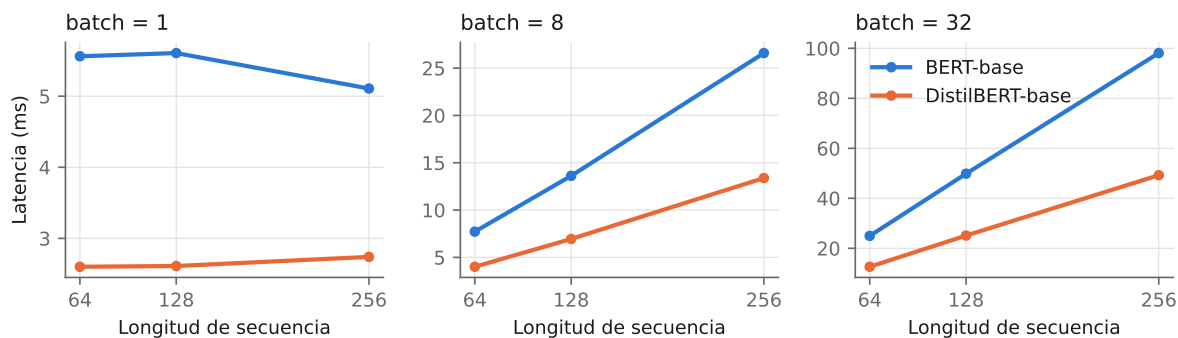


Figura 3: Latencia frente a longitud de secuencia, un panel por tamaño de *batch*. Se usan paneles separados en lugar de un solo gráfico porque las latencias de *batch* 1 y *batch* 32 difieren en un orden de magnitud y compartir eje aplastaría la curva inferior.

Los dos regímenes. La Figura 3 muestra además por qué los números por corrida no servían: con *batch* = 1 la latencia es plana respecto de la longitud de secuencia: BERT marca 5,56, 5,61 y 5,11 ms para 64, 128 y 256 *tokens*, y DistilBERT 2,60, 2,61 y 2,74. Multiplicar por cuatro la longitud no cuesta nada porque la GPU no está calculando: está esperando a que se lancen los *kernels*. Solo a partir de *batch* 8 la latencia empieza a seguir a la longitud, que es cuando la medida significa algo.

La conclusión práctica es que la ventaja de DistilBERT se cobra en los dos regímenes, pero por motivos distintos: menos *kernels* que lanzar cuando se sirve petición a petición, y la mitad de cálculo cuando se procesa por lotes.

La memoria no escala igual que los parámetros. El ratio de memoria pasa de $0,62\times$ con *batch* 1 a $0,77\times$ con *batch* 32 y 256 *tokens*. Con *batch* pequeño la memoria está dominada por los pesos, que sí siguen la proporción de parámetros; al crecer el *batch*, las activaciones pasan a dominar, y esas dependen de la longitud de secuencia y del tamaño del lote — iguales para los dos modelos — más que del número de capas. En despliegues con lotes grandes, la ventaja de memoria de DistilBERT se estrecha.

6.4. Las tres dimensiones a la vez

La Figura 4 condensa el argumento del informe. Los puntos de DistilBERT están a la izquierda (menos parámetros) y en la misma banda vertical que los de BERT (*accuracy* comparable), con burbujas de área similar o menor. La distancia vertical entre las dos columnas es pequeña en todos los casos y prácticamente nula en AG News.

7. La configuración ganadora en los dos modelos

El último paso aplica la configuración ganadora del estudio de ablación — **lineal**: cabeza Linear($768 \rightarrow C$), *encoder* entrenable — a BERT y a DistilBERT con el mismo protocolo y los mismos datos.

Con la cabeza lineal, BERT gana en los tres datasets: por 1,95 puntos en SST-2, 0,43 en Yelp y 0,11 en AG News. Es el mismo patrón de las corridas base — la distancia entre los dos modelos depende de la tarea y se estrecha casi hasta desaparecer en clasificación de temas — solo que ahora BERT también gana en AG News, aunque por un margen que sigue dentro del ruido.

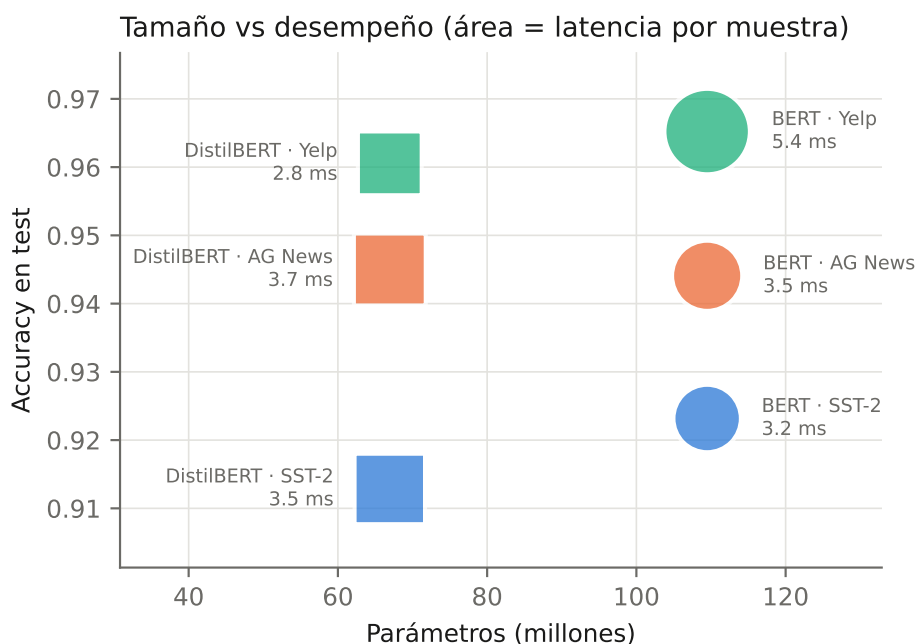


Figura 4: Parámetros frente a *accuracy* de test; el área de cada burbuja es la latencia por muestra. Resume el intercambio completo: cuánto ocupa, cuánto acierta y cuánto tarda cada combinación de modelo y tarea.

Dataset	Modelo	Accuracy	Precision	Recall	F1	F1 val.	Train (min)
SST-2	BERT	0,9289	0,9294	0,9286	0,9288	0,9541	4,2
SST-2	DistilBERT	0,9094	0,9099	0,9091	0,9093	0,9483	1,7
AG News	BERT	0,9453	0,9456	0,9453	0,9453	0,9481	9,0
AG News	DistilBERT	0,9442	0,9443	0,9442	0,9442	0,9462	5,4
Yelp	BERT	0,9653	0,9653	0,9653	0,9653	0,9661	16,0
Yelp	DistilBERT	0,9610	0,9610	0,9610	0,9610	0,9603	8,2

Tabla 10: Configuración *lineal* en los dos modelos. BERT tiene 109,5 M de parámetros y DistilBERT 66,4 M.

Un hallazgo secundario que merece atención. La cabeza *lineal* *mejora* a BERT respecto de su propia corrida base: 0,9289 frente a 0,9232 de *accuracy* en SST-2. La corrida base usa la cabeza por defecto de HuggingFace, que en BERT pasa el [CLS] por un *pooler* con *tanh* más *dropout*. Es decir: ese preprocesado adicional no aporta nada en estas tareas y en SST-2 incluso estorba. Es coherente con la conclusión del estudio de ablación — la capacidad que importa está en el *encoder* — y llevado un paso más allá: no solo las capas extra de la cabeza son innecesarias, sino que la transformación no lineal por defecto puede ser contraproducente.

8. Limitaciones

Cuatro, en orden de importancia para la interpretación de los resultados.

Una sola semilla. Todas las corridas usan la semilla 42. Sin repeticiones no hay estimación de la varianza, y por eso a lo largo del informe las diferencias por debajo de unas décimas se describen como empates en lugar de como victorias. La afirmación “la cabeza no importa” se apoya en que cinco configuraciones caben en 0,0014 de F1, lo cual es consistente con el ruido

esperable pero no es una prueba formal. Repetir con tres o cinco semillas y reportar media y desviación sería la mejora más valiosa a este trabajo, y es barata: unas 7,5 horas de GPU.

Presupuesto de dos épocas. El artículo de BERT recomienda entre 2 y 4 épocas para *fine-tuning*. Con 2 se favorece ligeramente al modelo que converge más rápido, y las curvas de SST-2 sugieren que ahí BERT aún tenía margen. Es un presupuesto idéntico para los dos modelos, así que no invalida la comparación, pero sí acota su alcance.

Yelp submuestreado. Se usan 100 000 de 560 000 ejemplos de train. Los valores absolutos de Yelp están por debajo de lo alcanzable con el conjunto completo; las diferencias *entre* modelos, que es lo que interesa, no deberían verse afectadas, porque ambos reciben exactamente el mismo subconjunto.

Registro de validación incompleto en las corridas base. Cinco de las seis corridas base se ejecutaron con una revisión anterior del código de entrenamiento que no serializaba el resumen de validación por época (`desempeno_val`) en el JSON. La *selección* del mejor *checkpoint* por F1 de validación sí se aplicó en todas — está en las tres revisiones del código —, y la validación se evaluó durante el entrenamiento, como demuestran los puntos de `val_loss` registrados en el historial. Lo que falta es solo el registro agregado, motivo por el cual la Tabla 4 no incluye columna de F1 de validación y la Tabla 10, que proviene de corridas posteriores, sí.

9. Conclusiones

1. **DistilBERT es la elección por defecto para estas tareas.** Conserva entre el 98,9 % y el 99,5 % de la *accuracy* de BERT con el 61 % de los parámetros, el 62 % de la memoria y la mitad de la latencia. La pregunta relevante no es si BERT es mejor — lo es, por poco — sino si ese margen justifica duplicar el costo de inferencia. En AG News no lo justifica en absoluto; en SST-2, donde la brecha llega a 1,95 puntos con la cabeza lineal, podría hacerlo si la aplicación es sensible a esos puntos.
2. **La brecha entre los modelos depende de la tarea, no es una constante.** Va de 0,11 puntos en AG News a 1,95 en SST-2. La clasificación de temas se resuelve en buena medida con vocabulario; el análisis de sentimiento sobre frases cortas, con negación y contraste, exige la composición que aportan las capas adicionales. Extrapolar un único número de “retención de desempeño” entre tareas sería un error.
3. **La capacidad que importa está en el *encoder*, no en la cabeza.** Cinco arquitecturas de cabeza — de un clasificador lineal a un MLP de dos capas ocultas — caben en 0,0014 de F1, mientras que congelar el *transformer* cuesta entre 3,4 y 8,4 puntos. El corolario práctico es que el esfuerzo de diseño debe ir a qué se adapta y con qué datos, no a la arquitectura del clasificador.
4. **La medición de eficiencia exige condiciones controladas.** La latencia registrada al final de cada entrenamiento sugería que BERT era más rápido que DistilBERT en SST-2. Medidos en la misma rejilla, mismo proceso y misma GPU, DistilBERT resulta 2× más rápido en los nueve puntos. La diferencia entre las dos mediciones no está en los modelos sino en el protocolo, y es un recordatorio de que una métrica de sistema sin condiciones explícitas no es un resultado.
5. **Congelar el *encoder* es un intercambio, no un error.** Entrena 0,6 M de parámetros en lugar de 66,4 M y tarda menos de la mitad. En AG News el costo es de 3,4 puntos de F1, que puede ser aceptable si el presupuesto de entrenamiento es el factor limitante.

Referencias

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [2] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *NeurIPS EMC² Workshop*, 2019.
- [3] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP*, pages 1631–1642, 2013.
- [4] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of ICLR*, 2019.
- [5] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

A. Reproducibilidad

A.1. Estructura del repositorio

```
src/
  paths.py          rutas del repo, ancladas a la raiz
  data_adapter.py   carga y normaliza SST-2, AG News y Yelp
  models.py         fabrica de modelos, cabeza configurable y congelado
  train.py          bucle de fine-tuning (escrito a mano, sin Trainer)
  metrics.py        accuracy/precision/recall/F1 + latencia, memoria, tamaño
  runs.py           consulta y seleccion de las corridas guardadas
  style.py          paleta y estilo comun de las figuras
  plots.py          figuras del informe (PDF + PNG)
  tables.py         tablas del informe, en Markdown
  benchmark.py      latencia y memoria en una rejilla controlada
scripts/           lanzadores de Slurm
results/
  metrics/          un JSON por corrida (nada se sobrescribe)
  figures/          figuras y tablas generadas por src/plots.py
  logs/            salida de los jobs de Slurm
docs/informe/      este documento
```

A.2. Cómo se reproduce

```
bash scripts/launch_base.sh distilbert # 3 corridas base
bash scripts/launch_base.sh bert      # 3 corridas base
CONFIGS=6 sbatch scripts/ablation.sbatch # 18 corridas del ablation
MODELO=bert CONFIGS=mejor MEJOR="lineal|--head-hidden none" \
  sbatch scripts/ablation.sbatch # 3 corridas de la ganadora con BERT
sbatch scripts/benchmark.sbatch # latencia y memoria controladas
python -m src.plots # figuras y tablas
```

Son 24 corridas, unas 2,5 horas de GPU en una RTX A6000, en serie porque la QOS del clúster permite una GPU a la vez.

A.3. Trazabilidad de las cifras

Ninguna cifra de este informe se transcribió a mano desde una consola. Cada entrenamiento escribe un JSON con nombre único (`{modelo}_{tarea}_{tag}_{fecha-hora}.json`) que contiene las métricas, la configuración completa (`vars(args)`), el historial de *loss* por iteración, el desglose por clase y la matriz de confusión. Nada se sobrescribe nunca. Las tablas y figuras se generan desde esos archivos con `python -m src.plots`, que produce además las versiones en Markdown de cada tabla en `results/figures/`.

Las etiquetas (`-tag`) distinguen el propósito de cada corrida: `base` para las del informe, `abl-<config>` para el estudio de ablación, y `smoke` o `pilot` para pruebas y exploraciones, que quedan excluidas automáticamente de las figuras.

A.4. Regenerar las figuras sin GPU

Los JSON están versionados, de modo que todas las figuras y tablas se reconstruyen sin GPU, sin `torch` y sin volver a entrenar:

```
pip install matplotlib numpy
python -m src.plots
```

La salida es determinista: dos ejecuciones consecutivas producen archivos idénticos byte a byte, de modo que un archivo marcado como modificado significa que algún dato cambió.

A.5. Compilar este informe

```
cd docs/informe
make          # usa tectonic; genera main.pdf
```

Las figuras se leen de `../../results/figures/` en su versión PDF (vectorial). El informe nunca contiene una copia de una figura, así que no puede quedar desincronizado respecto del *pipeline*.